

AWS Data Services & Governed Pipelines

PPOL5206 | Massive Data Fundamentals | Spring 2026

Last Week: The Serverless Pipeline



S3

Raw Files



Glue Crawler

Schema Discovery



Glue Catalog

Metadata



Athena

SQL Queries

SERVERLESS: You don't manage databases or servers
pure file-based analytics

But What's Missing?



What if an application needs thousands of transactions per second?

Athena is read-only analytics

→ RDS / Aurora



What if a dashboard requires sub-second response?

Athena is ad hoc, not real-time

→ Redshift



What if a dataset contains SSNs and not everyone should see them?

No access controls on the catalog

→ Lake Formation

The OLTP / OLAP Spine

	OLTP	OLAP
Purpose	Transactions	Analytics
Operations	INSERT, UPDATE, DELETE	SELECT, aggregations
Speed	Fast single-record	Fast full-table scans
Storage	Row-oriented	Column-oriented
AWS Service	RDS / Aurora	Redshift / Athena
Snowflake	Hybrid Tables (limited)	Core platform

Every service we cover today maps to one side of this divide.

Where Snowflake Fits (Revisited)

Cloud Services

Query optimization, metadata, security

Virtual Warehouses

Compute clusters, independent scaling

Centralized Storage

Columnar micro-partitions, S3-backed

Today's Question

How does the AWS-native stack compare to Snowflake, and when would an organization choose one over the other?

Platform choice is an organizational decision with cost, lock-in, and capability tradeoffs.

Where Transactions Live

Managed Relational Databases: OLTP in the Cloud

What "Managed" Means

Full Control / Full Responsibility ←————→ Less Control / Less Ops Burden

Self-Hosted (EC2 + PostgreSQL)

User manages: hardware, OS, patching, backups, replication, scaling, security, tuning

Amazon RDS

AWS manages: backups, patching, replication, failover
User manages: schema, queries, capacity planning

Amazon Aurora

AWS manages: all of the above + custom storage engine, auto-scaling storage, replication

Amazon RDS: The Managed Relational Database



Standard database engines hosted and managed by AWS

What AWS Handles

- ✓ Automated backups
- ✓ OS & engine patching
- ✓ Multi-AZ replication
- ✓ Automatic failover
- ✓ Monitoring & metrics

What You Still Manage

- Schema design
- Query optimization
- Capacity planning
- Application logic
- Security groups & access

The SQL covered in Weeks 2-3 works identically here.

RDS Engine Options and Why They Matter

PostgreSQL

Open-source, extensible, analytics-friendly

Startup choosing flexibility

MySQL

Most popular open-source RDBMS

Web apps, content management

Aurora (MySQL/PG)

AWS-optimized, compatible APIs

High-availability requirements

SQL Server

Microsoft ecosystem integration

Legacy .NET agency systems

Oracle

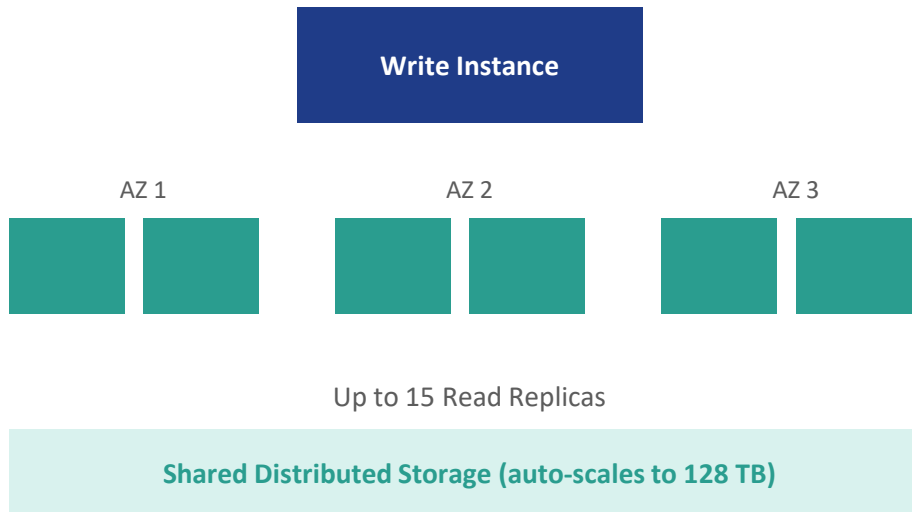
Enterprise features, licensing costs

"We're an Oracle shop"

Engine choice affects cost, licensing, migration complexity, and available features.

Amazon Aurora: RDS, But Faster

Aurora Distributed Storage Architecture



5×

MySQL throughput

3×

PostgreSQL
throughput

128
TB

Auto-scaling
storage

15

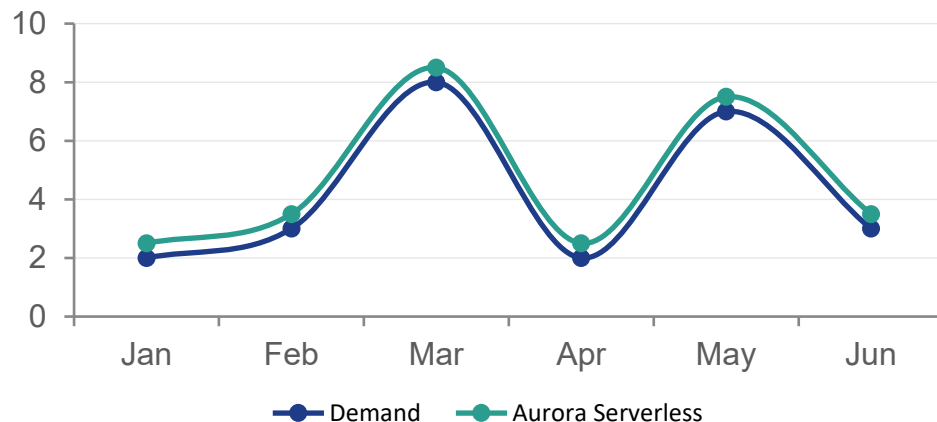
Read replicas

6

Storage copies
across 3 AZs

Aurora Serverless v2

Variable Workload → Automatic Scaling

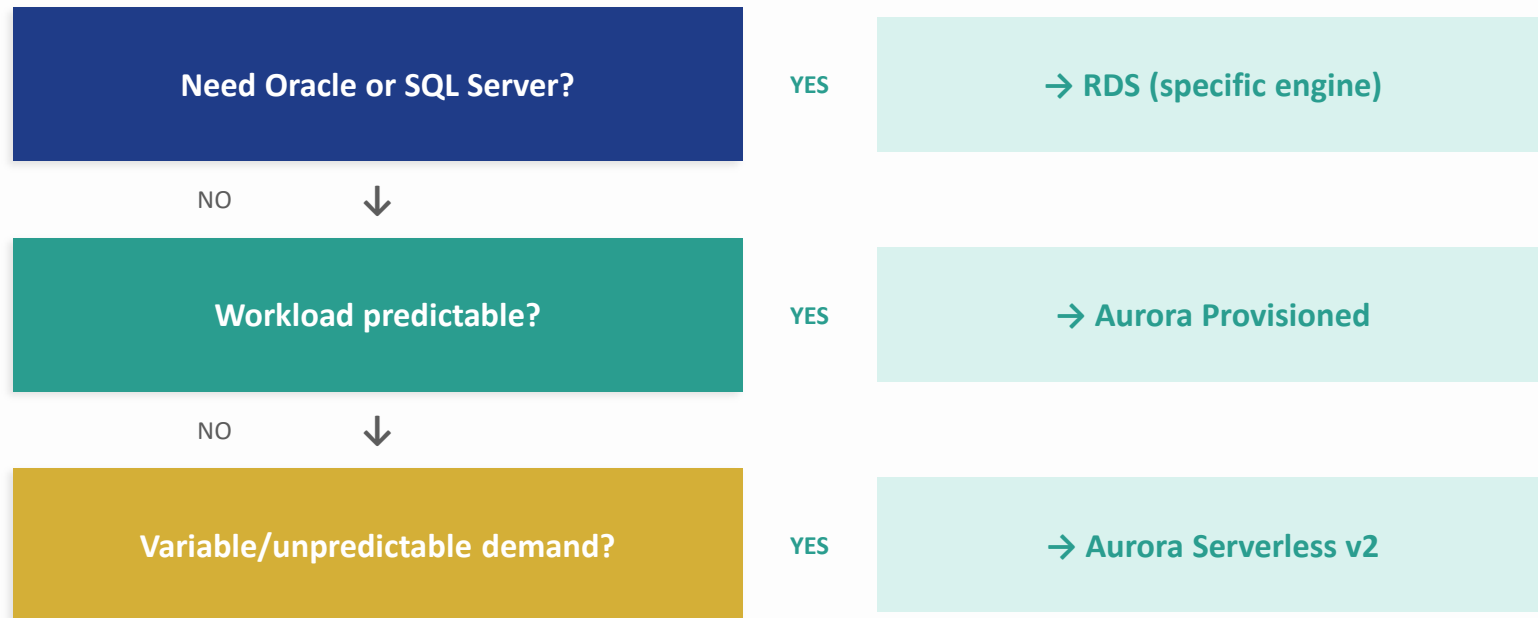


Key Features

- Scales in 0.5 ACU increments
- Pay for actual usage
- No capacity planning
- Instant scale-up on demand

Policy example: A grant application portal busy during submission periods, quiet otherwise; pay for use (with floor).

RDS vs. Aurora: Decision Framework



OLTP vs. OLAP Architecture



Example query: "Find all grants over \$50,000 awarded in 2024"

Row-Oriented (RDS/Aurora)

Reads ALL columns for every row to find matching records. Must scan through GrantID, Agency, Amount, Year, Status for each row.

- ✓ Fast for: UPDATE Grant #12345
- ✗ Slow for: Aggregate analytics

Column-Oriented (Redshift/Snowflake)

Reads ONLY Amount and Year columns. Skips GrantID, Agency, Status entirely. Much less data scanned.

- ✓ Fast for: Aggregate analytics
- ✗ Slow for: Single-row updates

Row vs. Column Storage Visualized

ROW STORE (RDS/Aurora)

G001	HHS	\$45,000	2024	Active
G002	DOE	\$72,000	2024	Complete
G003	EPA	\$28,000	2023	Active
G004	HUD	\$91,000	2024	Pending

UPDATE row 3 → read contiguous block

COLUMN STORE (Redshift/Snowflake)

G001	HHS	\$45,000	2024	Active
G002	DOE	\$72,000	2024	Complete
G003	EPA	\$28,000	2023	Active
G004	HUD	\$91,000	2024	Pending

SUM(Amount) WHERE Year=2024 → read only 2 columns

Physical storage layout determines which operations are fast. This is why OLTP and OLAP systems exist separately.

The Problem: Operational ≠ Analytical



Case Worker

Aurora handles benefit enrollment — fast, reliable transactions



Analyst Runs Big Query

3-year aggregation on production database consumes resources

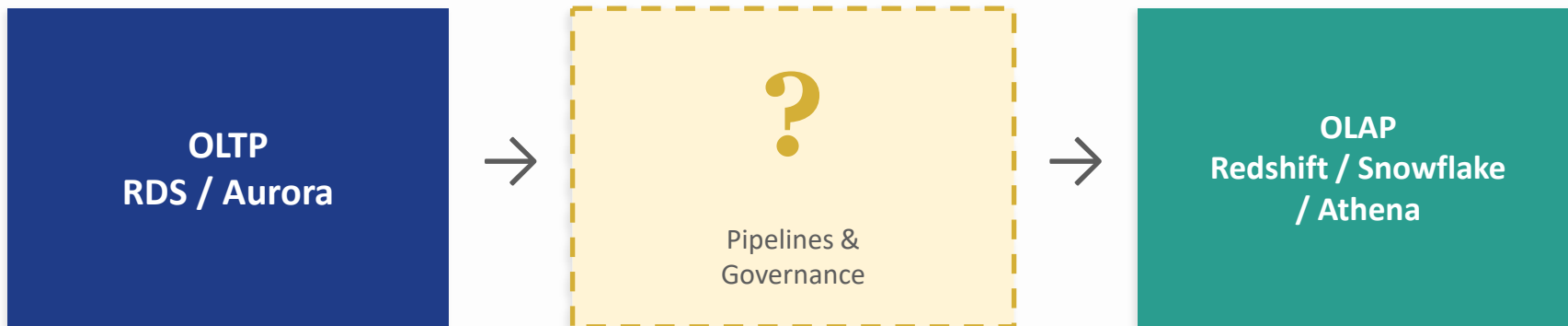


System Slows Down

Case workers experience delays, enrollment freezes

*"Don't run analytics on production." is the principle driving everything from Redshift through Zero-ETL.
Cloud services make shifting data structure easier*

The Operational → Analytical Pattern



The rest of today fills in this box and controls who gets to access the result.

Checkpoint: What We've Established

1. Operational systems need fast transactional databases (RDS / Aurora)
2. Analytical workloads need different architecture (columnar, parallel processing)
3. Data needs to flow from operational to analytical, with controlled access

Where Analytics Live

Cloud Data Warehousing:
columnar storage and massively parallel processing for analytical queries.

Amazon Redshift: The AWS Data Warehouse

AWS's answer to Snowflake: fast analytical queries on massive structured datasets

Columnar Storage

MPP Architecture

SQL-Based

Petabyte Scale

AWS Ecosystem

Deep AWS integration: IAM authentication, VPC networking, CloudWatch monitoring, Glue Catalog access, Lake Formation governance. Redshift Serverless is the recommended starting point for new workloads in 2026.

Redshift Serverless: Start Here

The Modern Entry Point

- ✓ No cluster management
- ✓ Auto-scales compute (RPU)
- ✓ Pay per query, not per hour
- ✓ Workgroup + Namespace model
- ✓ No nodes to size or tune
- ✓ Recommended for new workloads

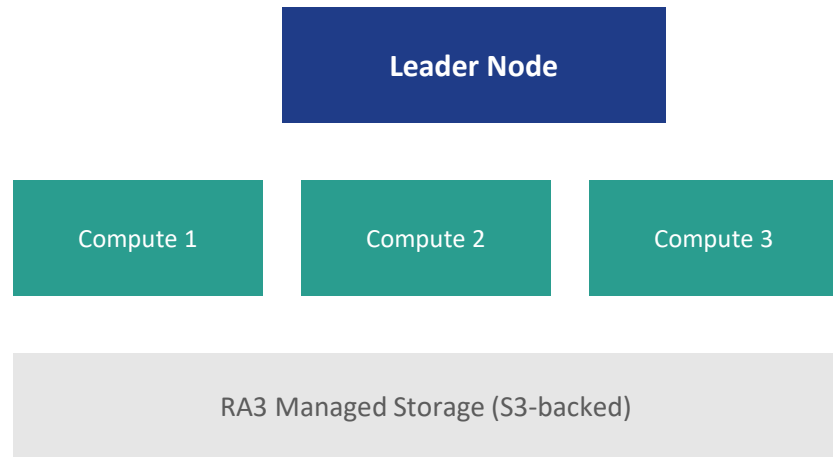
Cost in RPUs (Redshift Processing Units)

Serverless capacity measured in RPUs. Base: 8 RPU. Scales to 512 RPU automatically. You pay for RPU-seconds consumed while queries run.

Like Lambda for compute, you bring the SQL, AWS provides the infrastructure.

Industry shift: Serverless first, provisioned for mature workloads.

Redshift Provisioned: When You Need More Control



Tuning Controls

Distribution Keys

How data spreads across nodes

Sort Keys

How data orders within nodes

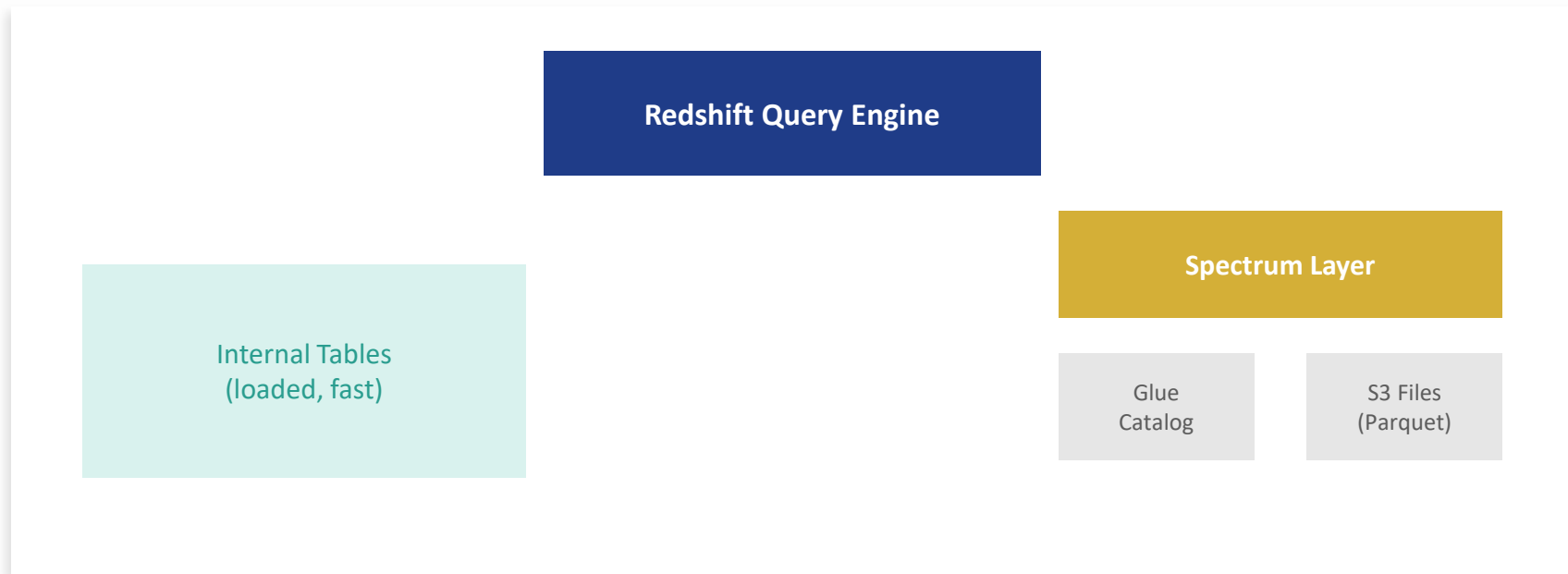
Concurrency Scaling

Auto-add clusters for spikes

Reserved Instances

Predictable cost savings

Redshift Spectrum: Query S3 Without Loading



The Glue Catalog you built in Week 6 - Redshift can read it too.

Redshift Security by Default (2025+)

Before 2025

- X Public endpoint by default
- X Encryption optional
- X Basic auth common
- X Misconfigurations → breaches

After 2025

- ✓ No public access by default
- ✓ Encryption enabled
- ✓ Secure connections enforced
- ✓ Security by design philosophy

AWS learned that many breaches came from misconfigured clusters. Secure defaults prevent human error.

Snowflake vs. Redshift: Architecture



Feature	Redshift	Snowflake
Compute/Storage	Separated (RA3)	Separated (day one)
Scaling	RPU-s or node resize	Virtual warehouses
Tuning	Dist/sort keys available	Fully abstracted
Pricing	RPU-sec or reserved	Credit-based
Ecosystem	AWS-native	Cloud-agnostic
Multi-cloud	AWS only	AWS, Azure, GCP

Architecturally converged: the choice is about ecosystem, control, and multi-cloud strategy.

Snowflake vs. Redshift: The Policy Lens



Agency A: All-in on AWS

Deep IAM integration, VPC peering, existing CloudFormation

→ Redshift

Agency B: Multi-cloud mandate

CIO requires vendor diversification, GovCloud + Azure

→ Snowflake

Agency C: Mixed teams

Data engineers on AWS, analysts prefer Snowflake UI

→ Both (via Iceberg)

Industry Change in Real Time

Redshift Python UDF Deprecation



Meta-lesson: Cloud services change.

Features get deprecated. Architectures evolve. Teams must plan for migration. This is why understanding concepts matters more than memorizing API calls.

Checkpoint: The Analytical Layer

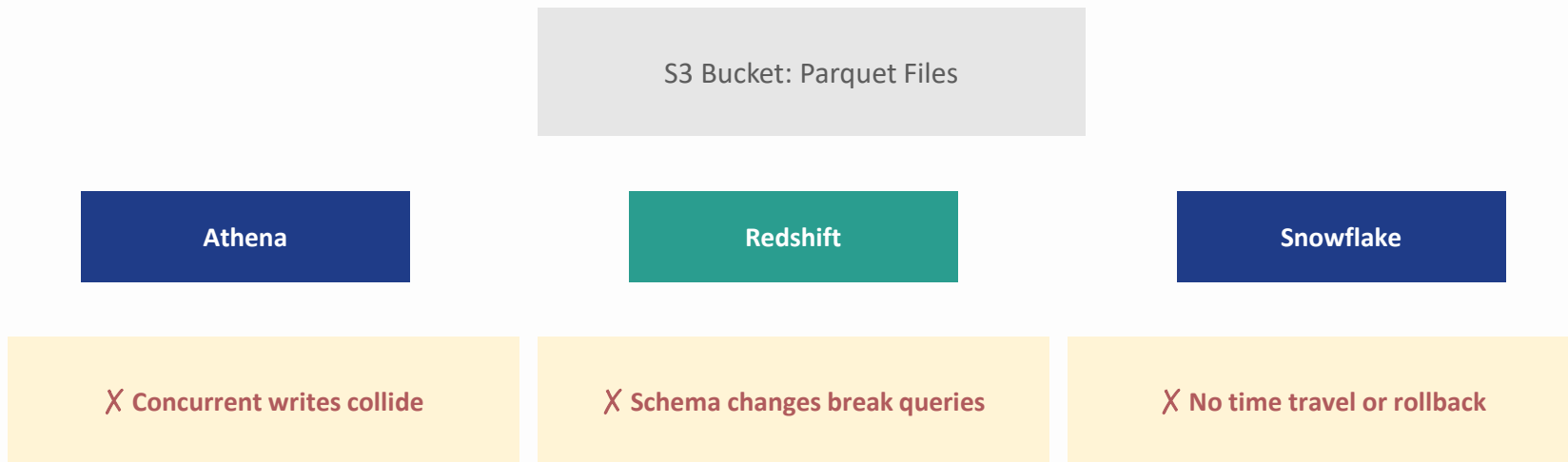
1. Redshift is the AWS cloud data warehouse; Snowflake the multi-cloud alternative
2. Both columnar, MPP, SQL-based (architectural convergence)
3. Choice is about organizational context: ecosystem, tuning control, multi-cloud strategy
4. The OLTP and OLAP pieces are in place: what connects them?

One Format to Rule Them All

Apache Iceberg: the open table format that connects everything.

The Problem Iceberg Solves

Scenario: Parquet files in S3 queried by three teams using different engines



Plain Parquet files have no transaction guarantees, no schema enforcement, no audit trail.

What Apache Iceberg Actually Is

Query Engines

Athena, Redshift, Snowflake, Spark

Iceberg Table Format

Metadata, snapshots, manifests, ACID transactions

Files in S3

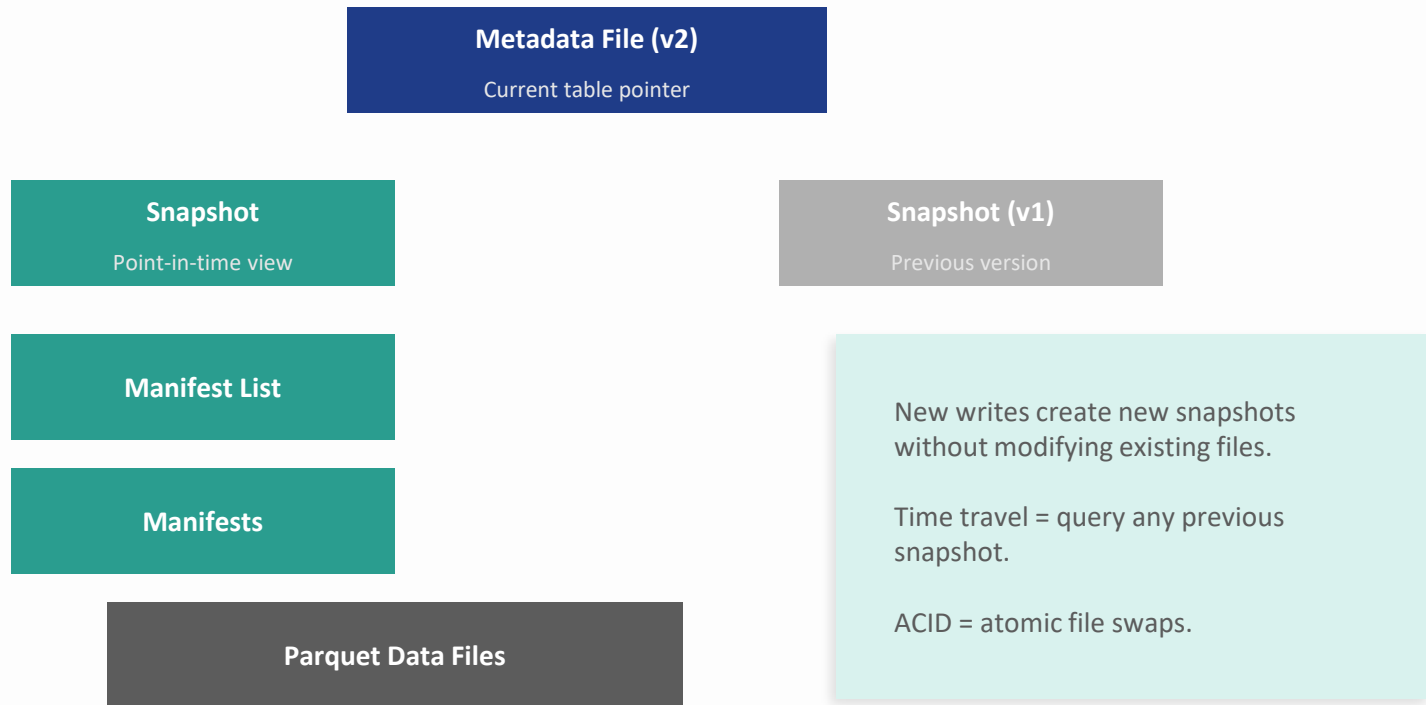
Parquet data files (unchanged)

OPEN STANDARD

- ✓ ACID transactions
- ✓ Schema evolution
- ✓ Partition evolution
- ✓ Time travel
- ✓ Hidden partitioning

Iceberg is a specification, not a database or a service.

Iceberg Metadata Architecture



Iceberg V3 — Active Evolution

Deletion Vectors

Mark rows deleted without
rewriting files
(faster deletes, less I/O)

Efficiency

Row Lineage

Audit trail tracking which rows
changed and when

FISMA audit

Table Encryption

Native encryption at the table
level for compliance

HIPAA, FedRAMP

Compliance features are not nice-to-haves, they're legal requirements: Iceberg is evolving to meet them.

Iceberg Everywhere: Industry Convergence

AWS

S3 Tables, Athena,
Redshift, Glue, EMR

Snowflake

Managed Iceberg,
Open Catalog/Polaris

**Apache
Iceberg**

Databricks

Unity Catalog

Open Source

Spark, Trino, Flink

Every major platform now supports Iceberg. Convergence reduces vendor lock-in.

AWS S3 Tables: Native Iceberg Storage

Before S3 Tables

- S3 bucket + manual Spark compaction
- External catalog management
- Manual metadata maintenance
- Multiple moving parts

With S3 Tables (2024-2025)

- Built-in automatic compaction
- Native catalog registration
- Lake Formation governance
- Glue Catalog integration
- AWS's bet on Iceberg

Snowflake & Iceberg: Open Catalog (Polaris)



OPEN INTEROPERABILITY

AWS makes S3 Iceberg-native. Snowflake makes its catalog Iceberg-compatible.

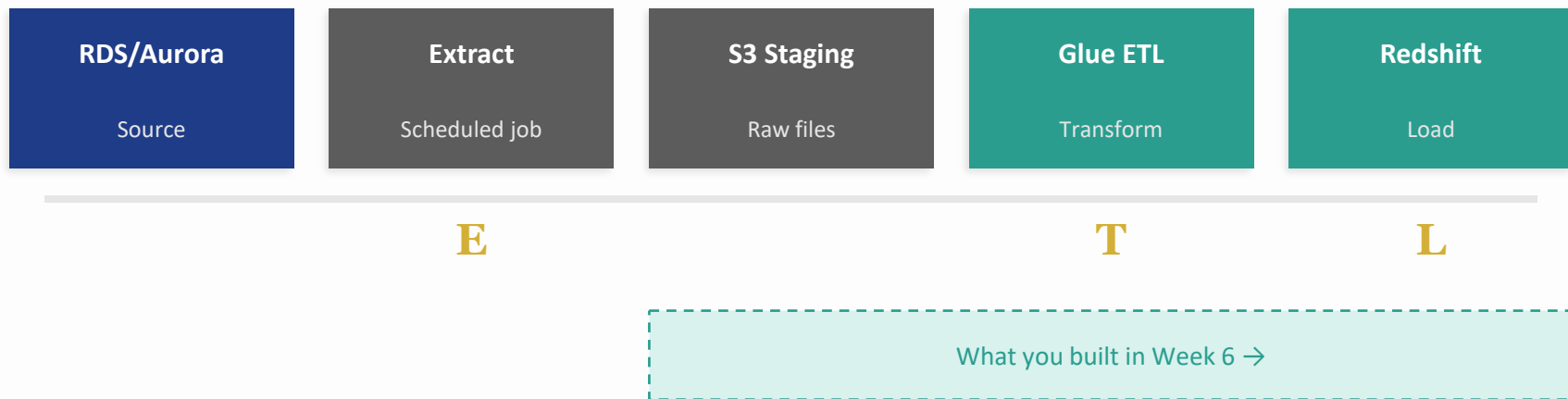
Both converging on the same open standard from different directions.

Bridging the Gap

Modern Data Pipelines:

*How data moves from operational to analytical systems,
and how the industry is automating it.*

The Traditional ETL Pipeline



The Pain Points of Traditional ETL

Building and maintaining pipelines is expensive

Engineering time, monitoring, on-call rotations

Pipelines break on schema changes

A new column upstream can cascade failures

Data only as fresh as the last run

Often daily → 12-24 hours stale

Debugging quality issues across stages is hard

Which stage introduced the error?

Someone gets paged at 2 AM when the job fails

Pipeline reliability is a staffing cost

Zero-ETL: AWS's Answer

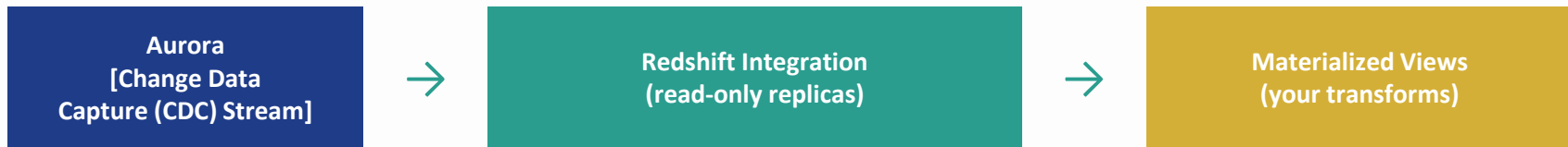
Aurora / RDS

Zero-ETL: automatic, near real-time

Redshift
Serverless

- ✓ No pipeline to build or maintain
- ✓ Data appears within seconds
- ✓ Aurora MySQL, Aurora PG, RDS PG supported
- ✓ Fully managed CDC replication

Zero-ETL Architecture Detail



E and L are automated. T happens inside Redshift via SQL.

Destination tables are read-only replicas of the source. Transformations happen via materialized views or downstream queries — you write SQL, not pipeline code.

Zero-ETL: Tradeoffs and Limitations

Advantages

- ✓ Near real-time data freshness
- ✓ No pipeline to maintain
- ✓ Simple setup and configuration
- ✓ Automatic schema sync

Limitations

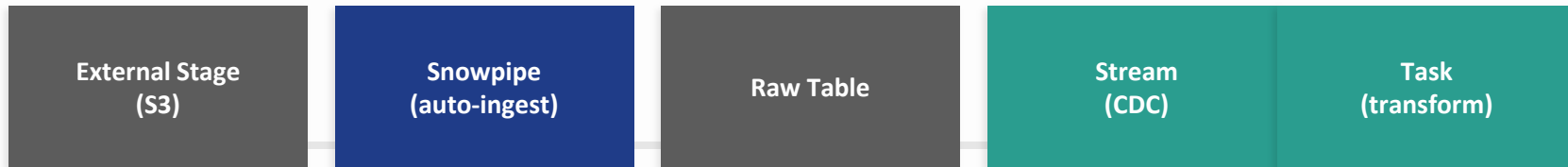
- ⚠ Continuous RPU cost
- ⚠ Read-only destinations
- ⚠ Schema changes → full resync
- ⚠ Limited engine support
- ⚠ No complex cross-source joins

Not a silver bullet - a genuine tradeoff between simplicity and flexibility.

Pipeline Pattern Comparison

	Traditional ETL	Zero-ETL	S3-Based ELT
Path	RDS→S3→Glue→Redshift	Aurora→Redshift	RDS→S3→Snowflake
Freshness	Hours (batch)	Seconds (CDC)	Minutes (Snowpipe)
Maintenance	High	Low	Medium
Flexibility	Any transform	SQL only	SQL + tasks
Cost Model	Per-job	Continuous RPU	Credit-based
Best For	Complex transforms	Simple replication	Multi-engine access

Snowflake: Streams, Tasks, and Snowpipe



Same problem, different products — recognize the pattern

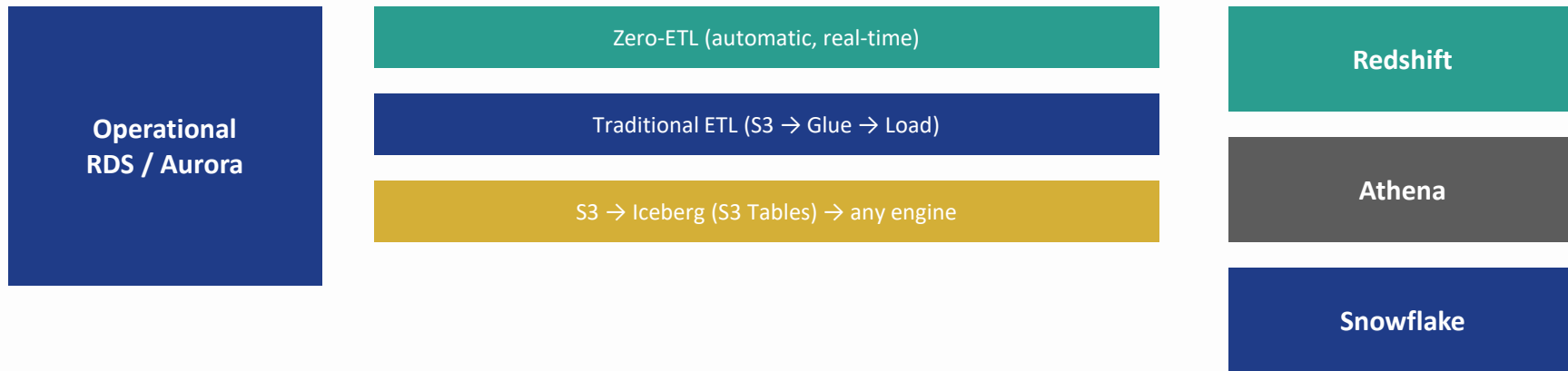
Streams = CDC (like Zero-ETL's change capture)

Tasks = Scheduled SQL (like Glue jobs)

Snowpipe = Continuous ingest from S3 (like Zero-ETL's auto-replication)

No "Zero-ETL" in the AWS sense, but similar outcomes via different mechanisms.

Operational → Analytical: The Full Picture (2026)



Multiple paths exist — choice depends on freshness, complexity, and platform strategy.



Zero-ETL: Simplest, real-time



Traditional ETL: Most flexible



Iceberg/cross-platform:
Least lock-in

Checkpoint: Pipelines

1. Data needs to move from OLTP to OLAP; traditional ETL gives control but costs effort
2. Zero-ETL automates the common case; Iceberg provides open format for cross-platform queries
3. But once data is queryable ... who gets to see it?

The Permission Problem

Data Governance: Who Gets to See What?

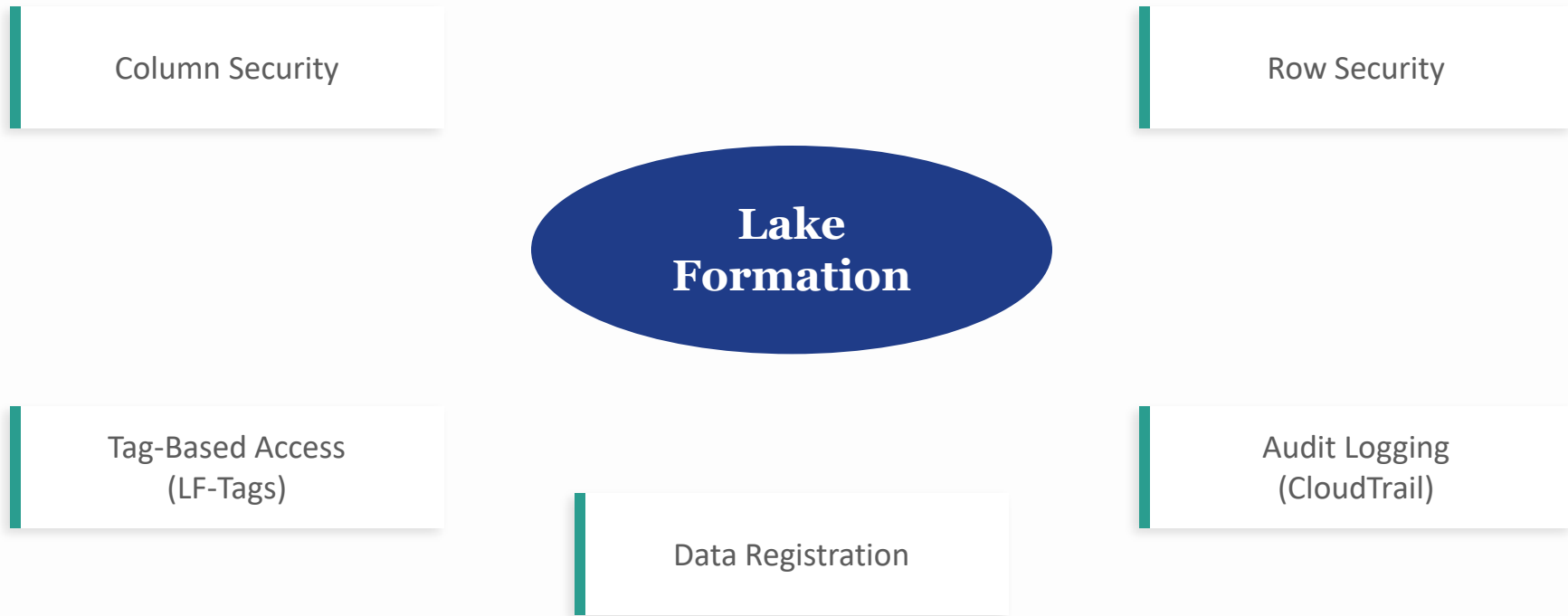
The Privacy Concern

```
SELECT * FROM census_microdata;  
  
-- Anyone with Athena access can run this right now
```

Name	SSN	Address	Income	County
Jane Smith	123-45-6789	123 Main St	\$52,000	Montgomery
John Doe	987-65-4321	456 Oak Ave	\$78,000	Prince George's

WHO ELSE CAN SEE THIS? No column-level filtering. No row restrictions. No audit trail.

AWS Lake Formation: What It Does



Database-style permissions applied to your data lake.

Fine-Grained Access Control Visualized

Analyst

Restricted	Restricted	Restricted	County Sample data	Status Sample data
-----------------------	-----------------------	-----------------------	--	--

Supervisor

Name Sample data	Restricted	Restricted	County Sample data	Status Sample data
--	-----------------------	-----------------------	--	--

Auditor

Name Sample data	SSN Sample data	Income Sample data	County Sample data	Status Sample data
--	---------------------------------------	--	--	--

One policy → enforced across Athena, Redshift Spectrum, EMR

LF-Tags: Permissions at Scale

PII: SSN, Name, Address

Public: County, Program Type

Confidential: Income, Benefits

How It Scales

Instead of table-by-table grants, tag data with attributes and grant access by tag.

New tables with the same tags automatically inherit permissions.

Scales to thousands of tables without per-table configuration.

This is how large government data catalogs manage access at scale.

Lake Formation Best Practices

Do

- ✓ Use LF-Tags for scalable management
- ✓ Register data locations explicitly
- ✓ Limit Lake Formation admin count
- ✓ Plan permission model before migration
- ✓ Test with restrictive roles first

Don't

- X Grant permissions table-by-table
- X Use Service Linked Role in production
- X Give everyone admin access
- X Migrate from IAM-only without planning
- X Start with open access and restrict later

The Bigger Picture: SageMaker Lakehouse

SageMaker Lakehouse

**Lake Formation
(Governance)**

**SageMaker Catalog
(Discovery)**

**S3 Tables
(Iceberg Storage)**

**Redshift
(Analytics)**

2025-2026 Direction

Lake Formation isn't being replaced — it's being embedded in a larger platform.

Built-In vs. Bolt-On Governance



Snowflake: Built-In

Roles, row access policies, dynamic data masking, object tagging, all within the platform.

Simpler for single-platform environments.

One platform, one governance model

AWS: Bolt-On (Lake Formation)

Separate service layered on top of Glue Catalog, connecting to Athena, Redshift, EMR.

Works across multiple engines.

Multi-engine, one governance layer

Neither is inherently better, it depends on the architecture.

Policies Organizations Work Under



FERPA

Student records: column-level masking of PII



HIPAA

Health data: PHI access controls required



CJIS

Criminal justice: strict authentication requirements



Title 13

Census: data cannot be re-identified

You will implement these controls. This is your competitive advantage.

Governance Checkpoint

1. Lake Formation provides fine-grained access control and audit logging across AWS analytics engines
2. Snowflake provides similar capabilities natively within its platform
3. Industry moving toward unified governance across lakes, warehouses, and engines
4. Next week: the regulatory and ethical frameworks driving these requirements

Putting It Together

Synthesis, decision frameworks, and lab setup.

The Full AWS Data Architecture (2026)

Governance: Lake Formation + IAM

Analytical: Redshift | Athena | Snowflake ❄️

Pipeline: Zero-ETL | Glue ETL | Snowpipe

Storage: S3 | S3 Tables (Iceberg) | Glue Catalog

Operational: RDS | Aurora | Aurora Serverless

Everything from the last two weeks in one diagram. The mental model to carry forward.

Decision Framework: Choosing Your Stack

Just query files?	→	Athena
Managed transactional DB?	→	RDS / Aurora
Cloud data warehouse?	→	Redshift Serverless or Snowflake
Real-time replication?	→	Zero-ETL
Fine-grained access controls?	→	Lake Formation
Cross-platform access?	→	Iceberg + catalog federation

What's Changed Since Last Year

	Zero-ETL GA for RDS PostgreSQL	2025
	Redshift security-by-default	2025
	S3 Tables (native Iceberg)	2024-2025
	Apache Iceberg V3	2025
	SageMaker Lakehouse rebrand	2025
	Snowflake Open Catalog (Polaris)	2024-2025
	Catalog federation across platforms	2025

Cloud technology changes quickly

Lab Preview

1

Explore RDS PostgreSQL

Connect, INSERT/SELECT, contrast with Athena

~15 min

2

Query in Redshift Serverless

Query Glue Catalog data, compare execution

~20 min

3

Lake Formation Permissions

Restrict columns, verify enforcement

~15 min

4

Snowflake Side-by-Side

Same query, compare syntax and execution

~15 min

5

boto3 Programmatic Access

List Glue databases, describe instances

~10 min

Lab Resources and Links

Lab Document

[Canvas](#) → [Week 7](#) → [Lab 7](#)

AWS Console

console.aws.amazon.com

Snowflake Console

app.snowflake.com

Quick Reference Card

[Canvas](#) → [Week 7](#) → [Quick Reference](#)

AWS RDS Documentation

docs.aws.amazon.com/rds

boto3 Quickstart

boto3.amazonaws.com/v1/documentation

COMING NEXT

Week 8: Data Ethics, Governance & Regulations

The governance technology from today becomes the implementation layer for the regulations we study next.

GDPR

CCPA

FERPA

NIST

AI Gov